LECTURE 2 | CLASS 9 | STUDENT NOTES

Object Oriented Programming

The Four Core Principles

Created by - Sourabh Gorai

Encapsulation | Abstraction | Inheritance | Polymorphism

The 4 Pillars

What is Object Oriented Programming?

Object Oriented Programming (OOP) is a way of writing computer programs by modeling them around real-world objects — just like how the real world works. Every object around you has two key things:

> • **Properties** — What it *is* (color, size, name, weight)
> • **Behaviors** — What it *does* (runs, plays, charges, moves)

For example, a **Dog** is an object. Its properties include name, breed, and color. Its behaviors include bark(), eat(), and run(). OOP lets us model such real-world objects directly inside a program, making code more organized, reusable, and easy to maintain.

The 4 Pillars of OOP

#	Principle	Core Idea	Real Life Analogy
1	Encapsulation	Wrap & Protect data	Medicine Capsule ■
2	Abstraction	Hide complexity	Car Dashboard ■
3	Inheritance	Reuse from parent	Child from Parents ■■■■■
4	Polymorphism	Same name, many forms	Person in roles ■

1 Encapsulation

Definition

Encapsulation means bundling the data (properties) and the methods (behaviors) of an object together into a single unit, and **restricting direct access** to some of that data from the outside world. In simple words — keep your data safe and protected, and only allow access through proper channels.

Real Life Examples

Example 1 — Medicine Capsule ■

> Inside a medicine capsule, there is a specific medicine powder. You cannot reach in and grab the powder directly — the capsule **protects it**. You take the whole capsule and your body processes it the right way. Similarly, in OOP, the data inside an object is protected. You use special methods called **getters** and **setters** to access or modify the data.

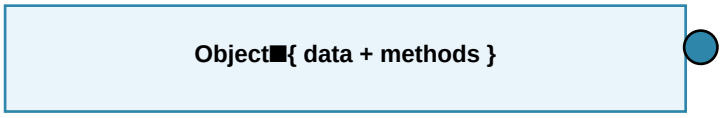
Example 2 — Your School Bag ■

> Your school bag contains your books, lunch box, and stationery. Not everyone can open your bag and take something out — **only you control** what goes in and what comes out. The bag wraps everything together and controls access.

Why is Encapsulation Important?

- Protects data from being accidentally changed
- Makes programs more secure and organized
- Restricts direct access to data
- Controls how data is accessed and modified

> ■ ***Encapsulation = Wrapping data + methods together + Protecting data from outside interference***



2 Abstraction

Definition

Abstraction means showing only the **necessary details** to the user and hiding the background complexity. You show WHAT something does — but you hide HOW it does it.

Real Life Examples

Example 1 — Driving a Car ■

> When you drive a car, you use the steering wheel, accelerator, and brake. You do NOT need to know how the engine combustion works, how brake pads apply friction, or how the transmission shifts gears. The car **abstracts all that complexity** and gives you a simple interface — pedals and a steering wheel.

Example 2 — TV Remote ■

> When you press the volume button on your remote, the volume increases. You do not see the infrared signals being sent, the circuit boards receiving them, or the code running inside the TV. The remote **abstracts all the internal complexity** and gives you simple buttons.

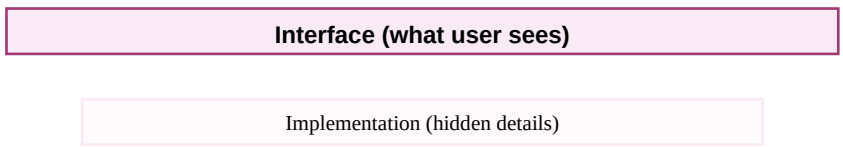
Example 3 — ATM Machine ■

> When you use an ATM, you enter your PIN and withdraw money. You do not see the banking servers processing your request, the security encryption, or the database updates. You only see a **simple screen with options** — everything complex is hidden away.

Why is Abstraction Important?

- Reduces complexity for the user
- Makes programs easier to use and understand
- Hides internal complexity from outside users
- Allows programmers to change internal logic without affecting users
- Provides a clean and simple interface

> ■ ***Abstraction = Show only what's necessary. Hide the complex internal details.***



3 Inheritance

Definition

Inheritance means one class (called the **Child Class** or Sub Class) can acquire the properties and behaviors of another class (called the **Parent Class** or Super Class). The child class automatically gets all the features of the parent class and can also add its own additional features on top.

Key Terms

Term	Also Called	Meaning
Parent Class	Super Class / Base Class	The class being inherited FROM
Child Class	Sub Class / Derived Class	The class that DOES the inheriting

Real Life Examples

Example 1 — Family Traits ■■■■■■■

> Your parents have certain traits — maybe your father is tall and your mother has brown eyes. You, as their child, inherit some of those traits — you may be tall *and* have brown eyes. You didn't start from scratch! You **inherited features from your parents**, and you also have your own unique personality on top of that.

Example 2 — Animals ■

Let's say we have a **Parent Class: Animal** with the following:

> Properties: name, age, color
>
> Behaviors: eat() sleep() breathe()

Child classes that inherit from Animal:

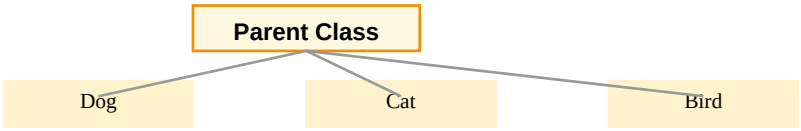
Child Class	Inherits From Animal	Its Own Addition
Dog ■	eat(), sleep(), breathe()	bark()
Cat ■	eat(), sleep(), breathe()	meow()
Bird ■	eat(), sleep(), breathe()	fly()

Why is Inheritance Important?

- **Code Reusability** — Write once, use in many classes
- **Less Repetition** — No need to rewrite the same code again and again

- Creates a **natural hierarchy**, just like the real world
- Easy to **maintain and update** — change the parent, and children update automatically

> ■ ***Inheritance = Child class gets the properties and behaviors of the Parent class. Reuse code. Extend features.***



4 Polymorphism

Definition

The word **Polymorphism** comes from Greek — *Poly* means **many** and *morph* means **form**. So Polymorphism means **Many Forms**. In OOP, Polymorphism means the **same method name can behave differently** depending on the object that is calling it.

Real Life Examples

Example 1 — A Person Playing Different Roles ■

> You are ONE person — but you behave very differently in different situations. At home with your parents, you are a respectful son or daughter. With your friends, you joke around and play. On the cricket ground, you focus and compete. In the classroom, you listen and take notes. **Same person. Different behavior depending on the situation.** That is Polymorphism!

Example 2 — Animals Making Sounds ■

Each animal has a method called **makeSound()** — but the output is different:

Object	Method Called	Output
Dog ■	makeSound()	"Woof!"
Cat ■	makeSound()	"Meow!"
Bird ■	makeSound()	"Tweet!"
Cow ■	makeSound()	"Moo!"

Example 3 — Drawing Shapes ■

> In a drawing app, every shape has a method called **draw()**. But Circle.draw() draws a circle, Rectangle.draw() draws a rectangle, and Triangle.draw() draws a triangle. **Same method name, completely different behavior for each object.**

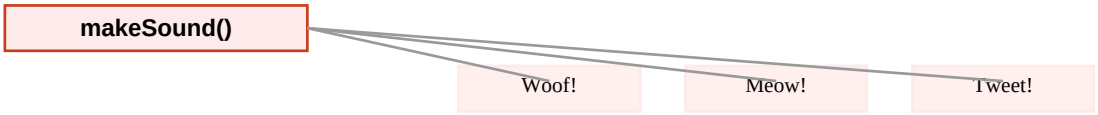
Two Types of Polymorphism

Type	Also Called	How it Works
Compile-time Polymorphism	Method Overloading	Same method name with different parameters in the same class
Runtime Polymorphism	Method Overriding	Child class redefines a method that already exists in the parent class

Why is Polymorphism Important?

- Makes code **flexible and scalable**
- One general instruction can work differently for many objects
- Makes programs **cleaner and easier to read**
- Reduces the need for multiple if-else conditions

> ■ ***Polymorphism = Same method name, different behavior depending on the object. Many Forms of the same action.***



Master Recap — All 4 Principles at a Glance

Principle	One-Line Definition	Real Life Example	Keyword
Encapsulation	Wrap & protect data inside an object	Medicine Capsule, School Bag	PROTECT
Abstraction	Show only what's needed, hide complexity	Car Dashboard, ATM, Remote	SIMPLIFY
Inheritance	Child gets features of parent class	Family traits, Animal → Dog	REUSE

Polymorphism	Same method, different behavior per object	Person in roles, Animal sounds	MANY FORMS

Remembering OOP with a School Example ■

Encapsulation	Every student's data (marks, address, phone) is kept safely inside their own profile. Teachers access it only through proper grade reports — not directly.
Abstraction	Teachers only see the final grade report — not the internal database queries, calculations, or server logic that generated it.
Inheritance	A 'ScienceStudent' class and an 'ArtsStudent' class both inherit from a general 'Student' class — they share common features like name, rollNo, and attend(), and each adds their own.
Polymorphism	Every student has a method called study() — but a ScienceStudent studies differently (lab experiments) than an ArtsStudent (essays and literature). Same method, different behavior.